

SURVEY PAPER

Mid-Training of Large Language Models

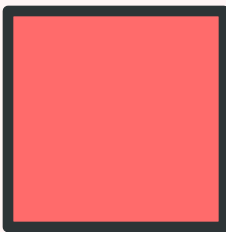
A Comprehensive Analysis of Data Distribution, Learning Rate Scheduling,
and Long-Context Extension Strategies



Data-Centric

Optimization

Context Extension



Contents

01

Introduction to Mid-Training

Understanding the critical continuation stage between pre-training and fine-tuning

02

Theoretical Foundations

Understanding why mid-training works through three key perspectives

03

Data Distribution Strategies

Comprehensive analysis of mid-training dataset composition and quality

04

Learning Rate Scheduling

Optimization strategies for stable convergence and efficient training

05

Long-Context Extension

Techniques for expanding model capabilities beyond initial training windows

06

Evaluation & Benchmarks

Measuring the impact of mid-training strategies across diverse tasks

07

Future Directions

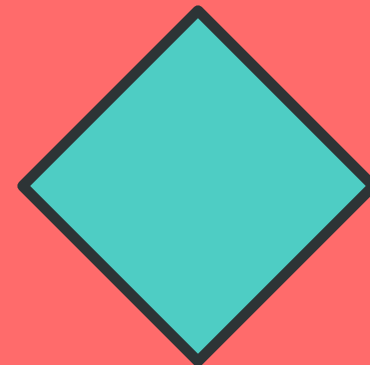
Open challenges and promising research avenues in mid-training optimization

30 Pages

Comprehensive Survey



01



Introduction to Mid-Training



Understanding the critical continuation stage between pre-training
and fine-tuning



What is Mid-Training?



Definition

Mid-training is a crucial continuation stage between general pre-training and task-specific supervised fine-tuning (SFT). It represents a paradigm shift from relying solely on web-scale corpora to adopting annealing-style phases that refine optimization dynamics and sharpen data quality.



Core Objectives

- ▶ Mitigate diminishing returns from noisy tokens in late training
- ▶ Stabilize convergence near favorable minima with reduced learning rates
- ▶ Expand capabilities in reasoning, coding, and long-context understanding



Key Insight

Without mid-training adjustments, models often **stagnate in capability development**, showing limited gains despite substantial additional compute investment.



Training Pipeline Evolution

1

Pre-Training

Web-scale corpora, broad coverage



2

Mid-Training ★

Annealing, curated data, context extension



3

Fine-Tuning (SFT)

Task-specific adaptation

15+
Models Analyzed

3
Core Domains



The Three Pillars of Mid-Training



Data Distribution

Shift toward curated or synthetic corpora emphasizing reasoning, coding, STEM, multilinguality, and instruction following.

- ✓ High-quality filtered web data
- ✓ Code & mathematical content
- ✓ Instruction-tuned & QA-style data
- ✓ Synthetic textbooks

★ **Impact:** Fosters compositional skills (chain-of-thought, math problem solving)



Learning Rate Scheduling

Adapt learning rate schedules with smoother decay to stabilize convergence and suppress gradient noise near promising optima.

- ✓ Linear or cosine decay
- ✓ Multi-stage schedulers (WSD)
- ✓ Warmup-stable-decay
- ✓ Power schedulers

★ **Impact:** Improves sample efficiency and enables finer assimilation of high-quality tokens



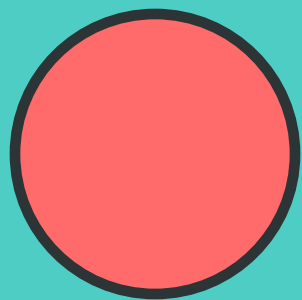
Long-Context Extension

Extend context length beyond 4K-8K limits via positional encoding remapping and progressive training curricula.

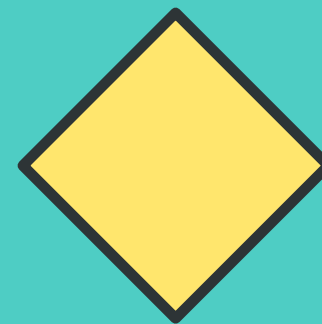
- ✓ Position Interpolation (PI)
- ✓ NTK-aware interpolation
- ✓ YaRN & ABF methods
- ✓ LongRoPE

★ **Impact:** Enables coherence across long documents and multi-file reasoning

Interdependence: Effective mid-training cannot be understood by analyzing each factor in isolation, but rather through their **combined impact** on model performance and efficiency.



02



Theoretical Foundations



Understanding why mid-training works through three key perspectives





Three Theoretical Perspectives

Core Rationale: Mid-training shifts the model's learning dynamics from **memorization** to **abstraction**



Gradient Noise Scale

The GNS reflects the amount of **useful signal per update step**. Higher-quality data induces greater gradient variance, yielding a higher GNS.

Key Mechanism:

- Helps models escape sharp minima
- Avoids overfitting to low-quality plateaus
- Improves optimization in late training

 **Insight:** Enhancing gradient signal-to-noise mitigates overfitting on noisy data

Better Optimization



Information Bottleneck

Representation learning can be interpreted as **compressing internal states** while retaining task-relevant information.

Key Mechanism:

- Reduces reliance on noisy features
- Emphasizes most predictive features
- Attenuates spurious correlations

 **Insight:** High-quality signals provide clearer, lower-entropy guidance

Clearer Representations



Curriculum Learning

Learning rate annealing aligns with **curriculum learning principles**—gradually shifting from diverse noisy corpora to challenging examples.

Key Mechanism:

- Early: diverse, noisy data for generalization
- Late: curated, informative examples
- Structured progression optimizes efficiency

 **Insight:** Reinforces complex skills like multi-step reasoning and code generation

Structured Learning

03

Data Distribution Strategies

Comprehensive analysis of mid-training dataset composition and quality



Types of Mid-Training Data



1. High-Quality Filtered Web Data

a

Curated subsets of web-scale corpora filtered for **quality, diversity, low toxicity, and educational value**.

FineWeb-Edu

DCLM

C4

RefinedWeb



2. Code & Mathematical Content

t

Structured content from **open-source repositories and mathematical Q&A pairs** to enhance symbolic reasoning.

Stack

OpenWebMath

FineMath

StackMathQA



3. Instruction-Tuned & QA-Style Data

a

Synthetic or curated question-answer pairs to enhance understanding of human intent and reasoning.

UltraChat

EvollInstruct

OpenOrca

FLAN



4. Synthetic Textbooks

Knowledge-dense synthetic data from knowledge graphs or advanced LLMs, providing high-quality educational content.

Cosmopedia

TinyGSM

Dolmino SynthMath



5. Long-Context Data

Long-form documents or Q&A spanning thousands of tokens, filtered from existing corpora or obtained via synthesis.

Books

Code Repos

DCLM



6. Reasoning & CoT Data

Chain-of-thought annotations providing explicit demonstrations of problem decomposition for structured reasoning.

Synthetic CoT

Web-filtered

GSM8K



7. Fill-in-Middle (FIM) Data

a

Enforces **bi-directional contextual reasoning** and improves data efficiency by predicting middle text based on context.



Mid-Training in Popular Models - Part 1

Q1 Qwen3

Stage 1: 30T tokens, 119 languages

Stage 2: 5T high-quality (STEM, coding, reasoning)

Stage 3: Long-context (75% 16k-32k, 25% 4k-16k)

Φ4 Phi-4

Stage 1: ~10T tokens, filtered web data

Stage 2: Synthetic tokens + ultra-filtered reasoning data

Mid-Training: 75B long-context + 175B recall tokens

L3 Llama3 405B

Stage 1: Initial
15T multilingual tokens

Stage 2: Long-Context
800B tokens

Stage 3: Annealing
40M tokens, upsampling HQ

Data Mix: 50% general knowledge | 17% code | 25% math/reasoning | 8% multilingual

AF AFM (Apple)

Stage 1: 6.3T tokens @ 4k

Annealing: 1T tokens @ 8k (downweight web, upweight code/math)

Long Context: 100B tokens @ 32k + synthetic QA

DS DeepSeek-V3

Pre-training: 14.8T diverse, high-quality tokens

Context Ext 1: 60B tokens @ 32K

Context Ext 2: 60B tokens @ 128K



Mid-Training in Popular Models - Part 2

S2

SmolLM2

S1: 6T (90% web, 10% code)

S2: 2T (75% web, 20% code, 5% math)

S3: 2T (74% web, 16% code, 10% math)

Anneal: 1T HQ (58/24/14/4%)

Long: 75B (40% long docs)

N4

Nemotron-4

General: 8T tokens

Mid-Train: 1T tokens

Mix: 70% EN, 15% multi, 15% code

Two distributions: reweighted tokens + QA-style

O2

OLMo 2

Stage 1: ~5T web data

Mid-Train: Dolmino Mix 1124

Smaller but higher-quality + synthetic

7B: 3 runs × 50B

13B: 3×100B + 1×300B

Z1

Zamba-v1

Initial: 950B open web

Math: StackMathQA

Mid-Train: 5T (60% pretrain + 40% HQ) **Code:** EvolInstructCode

Instruct: OpenOrca

M1

MiniMax-M1

Training: 7.5T tokens from Minimax-Text-01

Data Quality: Refined parsing, semantic dedup, prioritizing QA pairs

Composition: 70% STEM, code, books, reasoning

LR: 8e-5 → 8e-6 over 5T

PP

Pangu Pro MoE

Phase 1: General

9.6T tokens, foundational knowledge

Phase 2: Reasoning

3T tokens, STEM + synthetic CoT + 32K context

Phase 3: Annealing

0.4T tokens, 20% instruct, 18% STEM



Key Insights for Data Selection

1

Quality Over Quantity

Unlike early pre-training, mid-training **emphasizes quality**. Filtered corpora consistently outperform unfiltered ones. Different datasets have complementary strengths.

Example: FineWeb-Edu excels on MMLU, DCLM on commonsense

2

Prioritize Educational, Coding, Math Data

Consistent gains by **replacing low-signal web text** with structured educational, math, and coding data.

Evidence: SmoLLM2 ablations show strong STEM gains

3

Instruction Data Timing Uncertain

Models report **both benefits and limitations**. Upsampling helps reasoning, but optimal placement remains unresolved.

Open Question: Early vs. late introduction in pipeline

4

Long-Context Requires Balance

Training **solely on long data can hurt performance**. A balanced mixture with high-quality short-context data is critical.

Best Sources: Books and code repositories

5

Domain Data: Small but Mighty

Even a **small fraction of domain-specific data** (math, coding) produces disproportionate gains.

Principle: Quality-over-quantity at this stage

6

Synthetic Data Enhances Efficiency

Synthetic corpora can be **tailored to emphasize underrepresented domains**, compensating for limitations in natural text.

Value: Amplifies learning efficiency in mid-training

7

Reasoning Data Critical for STEM

FineMath4+ achieves **2* on GSM8K and 6* on MATH** vs. baseline, showing importance of step-by-step reasoning.

Mechanism: Explicit problem decomposition patterns

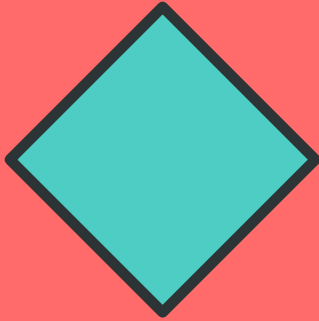
8

Maintain Distributional Continuity

Mismatch between mid-training and pretraining corpora can lead to **catastrophic forgetting**.

Strategy: OpenCoder retains 84% from original distribution

★ **Bottom Line:** Mid-training is a **targeted specialization phase**, leveraging focused, high-quality data to enhance core reasoning and coherence.



04

Learning Rate Scheduling



Optimization strategies for stable convergence and efficient training





Key Factors Affecting LR Scheduling



Warmup Setting

The **warmup phase** mitigates optimization instability during early iterations, while the decay phase governs convergence dynamics.

Benefits:

- Reduces loss sharpness
- Improves optimization conditioning
- Enables larger peak LR

Challenge: Optimal duration varies across model scales



Decay Strategy

A large initial LR suppresses memorization of noisy data, while gradual decay facilitates learning complex patterns.

Common Approaches:

- Cosine decay (most popular)
- Linear decay
- Multi-stage schedulers

Risk: Overly steep decay may halt learning prematurely



Batch Size

Increasing batch size lowers variance of stochastic gradient estimates, supporting the use of higher learning rates.

Relationship:

- Larger batch → Lower gradient variance
- Supports higher LR
- Requires proper warmup

Insight: Large mini-batch needs cautious initial rate



The Great Debate:

Kaplan et al. claim final performance is largely insensitive to schedule shape vs. Hoffmann et al. showing substantial influence on optimization dynamics.



Common LR Scheduler Types - Part 1

1

Composite Structure: Most pipelines use **linear warm-up** followed by a **decay phase** (cosine, linear, or multi-stage)

1 Linear Scheduler

LR **decreases linearly** from peak to end value over decay steps.

$$\eta_t = \eta_{\text{peak}} - (\eta_{\text{peak}} - \eta_{\text{end}}) \cdot (t / S_{\text{dc}})$$

+ Simple and effective

- May decay too fast

2 Cosine Scheduler

LR **follows cosine curve** after warmup. Most widely adopted in LLMs.

$$\eta_t = \eta_{\text{end}} + \frac{1}{2}(\eta_{\text{peak}} - \eta_{\text{end}})(1 + \cos(\pi t / S_{\text{dc}}))$$

+ Smooth, gradual decay

+ Used in GPT-3, Gopher, Chinchilla

3 Exponential Scheduler

LR **decreases exponentially** by multiplying with fixed decay factor at each step.

$$\eta_t = \eta_{\text{init}} \cdot e^{(-kt)}$$

+ Smooth, continuous reduction

- Less control, may decay too fast

4 Knee Scheduler

An **explore-exploit strategy** with constant high LR followed by linear decay.

Two Phases:

- **Exploration:** Constant high LR
- **Exploitation:** Linear decay to zero

+ Wide-minima hypothesis

+ No extra hyperparameters



Common LR Scheduler Types - Part 2

5 Cyclical Scheduler

LR **varies cyclically** between boundary values rather than decaying monotonically.

$$\eta_t = \eta_{\text{init}} + (\eta_{\text{peak}} - \eta_{\text{init}}) \cdot \max(0, 1 - |t/s - 2\lfloor t/2s \rfloor|)$$

- + Adaptive, no extensive tuning
- + Near-optimal accuracy

6 WSD Scheduler

Warmup-Stable-Decay with three phases designed for two-stage pretraining.

Three Phases:

- **Warmup:** LR increases
- **Stable:** Constant max LR
- **Decay:** Rapid reduction

- + Efficient continual training
- + No predetermined steps needed

7 Power Scheduler

Extends WSD with **power-law relationship** for zero-shot LR transfer across hyper parameters.

$$\eta_{\text{power}}(n) = \min(\eta_{\text{peak}}, \beta \cdot a \cdot n^b)$$

- + Zero-shot LR transfer
- + Agnostic to batch size & tokens
- + Used by Granite 3.0

8 Multi-step Scheduler

Replaces cosine with **discrete drops** at fixed milestones, enabling efficient continual training.

Structure:

- Linear warmup
- Two discrete decay steps
- Earlier phases reusable

- + Facilitates checkpoint reuse
- + Used by DeepSeek



LR Schedulers in Recent LLMs

Overview of LR Schedulers in Representative LLMs

Model	LR Decay	LR Path	Optimizer	Staged
GPT-3 (175B)	Cosine	2e-4 → 2e-5	Adam	✗
Gopher (280B)	Cosine	1e-7 → 6e-4 → 6e-5	Adam	✗
Chinchilla (70B)	Cosine	1e-7 → 4e-5 → 4e-6	AdamW	✗
PaLM (540B)	Inv. sqrt	1e-7 → 1.25e-4 → 1.25e-5	Adafactor	✗
LLaMA (65.2B)	Cosine	0 → 1.2e-4 → 1.2e-5	AdamW	✗
LLaMA 2 (70B)	Cosine	0 → 1.5e-4 → 1.5e-5	AdamW	✗
LLaMA 3 (405B)	Cosine → Linear	0 → 8e-5 → 8e-7 → 0	AdamW	✓ (3)
DeepSeek (67B)	Multi-stage	0 → 2.4e-4 → 7.6e-5 → 2.4e-5	AdamW	✓
MiniCPM (2.4B)	WSD	0 → 0.01 → 0.01·f(s-T)	AdamW	✓
Granite 3.0 (8B)	Power	0 → 0.02 → 4β·f(n,N,Ndecay)	AdamW	✓
Phi-4 (14B)	Linear	0 → 3e-4 → 0	AdamW	✓
OLMo 2 (13B)	WSD	0 → 3e-4 → 3e-5 → 8e-6	AdamW	✓

Cosine Most Popular

Multi-Stage Rising

AdamW Dominates



Key Insights for LR Scheduling



Linear Warm-up + Cosine/Linear Decay is Widely Used

This scheme is the **most common choice** in LLM training due to empirical stability, even though its design is guided more by practice than theory.

Reality: Heuristic tuning remains prevalent in the field



Warm-up Helps Stability

Warm-up phases are known to **alleviate gradient instability** and loss sharpness in early training, facilitating the use of larger peak learning rates.

Challenge: Optimal duration lacks theoretical justification



Decay Strategies Remain Unsettled

The design of the decay phase is **far from resolved**. Effectiveness interacts intricately with batch size, model scale, and optimizer dynamics.

Debate: Linear vs. cosine vs. multi-stage



Scaling Laws Help LR Selection

Empirical scaling laws provide **practical guidance** for choosing learning rates across model sizes, though they remain incomplete and model-sensitive.

Kaplan et al.: Larger models need smaller LR

Li et al.: Fixed final LR → stable convergence



Critical Role but Unresolved Uncertainties

Current evidence highlights **both importance and open questions**. Conflicting empirical results show lack of consensus and reproducible benchmarks.

Conflicting Views:

- Kaplan: Insensitive to schedule shape
- Hoffmann: Significant variance across types



Emerging Trends

- ▶ Explicit annealing phases gaining popularity
- ▶ WSD and multi-stage schedulers rising
- ▶ Power schedulers for zero-shot transfer

05

Long-Context Extension

Techniques for expanding model capabilities beyond initial training windows



RoPE and Context Extension Fundamental



Self-Attention with RoPE

RoPE encodes positions by **rotating query/key vectors** in 2D sub-planes of the embedding space.

$$\theta_j = b^{-(2(j-1)/d)}, b = 10000$$

Each dimension rotates at rate θ_j



Key Property

Inner products of rotated vectors **depend only on relative positions**, not absolute positions.

Why This Matters:

- Models trained at one length can generalize to longer contexts
- Enables context extension via frequency remapping



Unified Formulation

All extension methods use a **frequency remapping function** $h(\theta_j)$ that rescales each base frequency:

$$f'(x_m, m, \theta_j) = f(x_m, m, h(\theta_j))$$

Scale Factor: $s = L_{\text{target}}/L_{\text{train}} > 1$



Wavelength Concept

$$\lambda_j = 2\pi / \theta_j = 2\pi \cdot b^{(2(j-1)/d)}$$

Wavelength determines how quickly position embeddings change:

- **Low frequency** (small θ): Long wavelength, slow rotation
- **High frequency** (large θ): Short wavelength, fast rotation

 **Core Idea:** By carefully choosing $h(\theta_j)$, we can make models generalize from training length L_{train} to target length L_{target} without retraining from scratch.



Five Context Extension Methods - Part

1

1

Position Interpolation (PI)

Rescales absolute positions when evaluating sequences longer than pre-training window.

$$hPI(\theta_j) = \theta_j / s$$



Conceptually simple, light tuning



Uniform compression hurts high-frequency signals

Characteristics:

- Uniform scaling
- All frequencies compressed equally
- May drop short-context quality

2

NTK-aware Interpolation

Inspired by NTK theory—extends low-frequencies more, high-frequencies less to preserve local information.

$$hNTK(\theta_j) = \theta_j \cdot s^{(-2j/d)}$$



Better locality retention



Minimal tuning needed



Hyperparameter coupling

3

NTK-by-parts

Piecewise approach preserving shortest wavelengths while still extending range.

Three Regimes:

- High freq ($\gamma=1$): No interpolation
- Low freq ($\gamma=0$): Full interpolation
- Mid freq: Weighted mix



Strong short-context preservation



More boundaries to tune



Five Context Extension Methods - Part 2

4

YaRN

Augments NTK-by-parts with **length-dependent temperature scaling** applied to attention logits.

Temperature Scaling:

$$t = \alpha(s)^{-2}, \alpha(s) = 0.1 \ln(s) + 1$$

Implementation:

$$q\text{YaRN} = \alpha(s) \cdot q\text{NTP}, k\text{YaRN} = \alpha(s) \cdot k\text{NTP}$$

- + Efficient route to 32k-128k
- + Robust convergence with few steps
- Extra temperature to tune

5

LongRoPE

Learns **non-uniform token-dependent scales** via search, applied progressively in stages.

Search-Based Optimization:

Minimize discrepancy between original and scaled logits

Key Features:

- Progressive extension stages
 - Short-length recovery mechanism
 - Anchor positions every ~512 tokens
- + Reaches ~2M context
 - + Recovers short-length performance
 - Complex pipeline (search + stages)

Unified Comparison

Method	$h(\theta)$	Pros	Cons
PI	θ/s	Simple, robust	Hurts high-freq
NTK	$\theta/s^p(\theta)$	Better locality	Hyperparam coupling
NTP	Piecewise	Clean trade-off	More knobs
YaRN	NTP + temp	Efficient, robust	Extra tuning
LongRoPE	Learned	~2M context	Complex pipeline



Context Extension in Popular LLMs

Context Extension at a Glance

Model	Max Len	Context Extension	Architecture
Nemotron-4 15B	4k	RoPE (native)	—
InternLM2	32k	ABF (RoPE 50K→1M)	GQA
MiniCPM-128K	128k	ABF + NTK-aware	—
Phi-3 Mini	128k	LongRoPE	Block-sparse Attn
DeepSeek-V2	128k	YaRN	MLA, MoE
Llama-3.1	128k	ABF + YaRN	GQA
Hunyuan-Large	256k	ABF	MoE
Qwen-2.5-Turbo	128k	ABF + YaRN + DCA	GQA, Minference
Qwen-2.5-1M	1M	ABF + YaRN + DCA	Sparse Attn
MiniMax-01	4M	ABF + YaRN + DCA	LightningAttn 7:1
Llama-4 Scout	10M	NoPE/iRoPE + temp	Interleave NoPE

ABF Most Popular

YaRN for 128k+

GQA/MLA Common

Progressive Growth



Key Paradigms for Long-Context



Frequency Remapping Unifies Methods

Long-context schemes are modeled by **two independent controls**: (i) frequency remapping $h(\theta; s)$ and (ii) optional temperature scaling $\alpha(s)$.

All methods: PI, NTK, NTP, YaRN, LongRoPE are specific choices of $h(\cdot)$ and $\alpha(\cdot)$



Trade-off: Global Reach vs. Local Fidelity

Pure interpolation stretches all frequencies equally, potentially blurring local cues. NTK-aware methods spare high-frequency bands.

Pure Interpolation
Equal stretch, may blur

NTK-aware
Preserves short-range



Architecture Assists Matter but Are Orthogonal

Interleaved global/local attention, GQA/MLA, and sparse attention primarily **reduce KV-cache cost**; they enable long sequences but don't replace frequency remapping.



Progressive Expansion is Robust

Effective recipes expand the window **progressively** ($4k \rightarrow 32k \rightarrow 128k$) with:

- Annealed learning rate
- Mixed long/short tokens (30-60% long)
- Synthetic tasks (FIM, retrieval)



Practical Default

For most use cases:

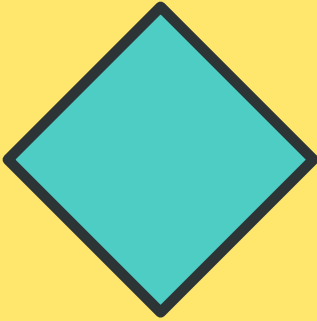
$L_{\text{target}} \leq 128k$: ABF ($\theta \approx 10^6$) + YaRN $\alpha(s)$

$L_{\text{target}} \geq 1M$: Staged ABF (θ up to 10^7 - 10^8) + LongRoPE



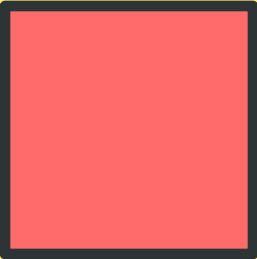
Key Takeaway

A robust recipe pairs **progressive ABF** with **non-uniform $h(\cdot)$** and mild $\alpha(s)$ schedule, while architecture assists unlock memory budget.



06

Evaluation & Benchmarks



Measuring the impact of mid-training strategies



Standard Evaluation Benchmark

At the mid-training stage, evaluation relies on **15 representative benchmarks** across six categories, ensuring comparability with prior work.



General

MMLU (5-shot)

MMLU-Pro (5-shot, CoT)

ARC (0-shot)

HellaSwag (0-shot)

BBH (3-shot, CoT)



Math & STEM

GPQA (5-shot, CoT)

MATH (4-shot, CoT)

GSM8K (4-shot, CoT)



Coding

HumanEval (0-shot)

MBPP (3-shot)

MultiPL-E (0-shot)



Multilingual

MGSM (8-shot, CoT)

Flores-101 (5-shot)



Long Context

LongBench

RULER



15

Total Benchmarks

6

Categories Covered



Mid-Training Uplift Summary

Mid-training improvements fall into **three major dimensions**, providing structured comparison across models.

A. Advanced Reasoning & Adaptability

Phi-4

Surpasses teacher on STEM-focused QA; meets Llama-3.1-405B on reasoning

DeepSeek-V2

88.9% on MiniF2F-test; solves 47/658 PutnamBench problems

Zamba

Achieves Llama2 performance with 1T tokens (vs. 2T)

MAP-Neo

Rapid code metric improvements during decay phase

B. Scalable Efficiency

MiniCPM4

Comparable to Qwen3 using 22% of tokens (8T vs. 36T)

Gemma2

Up to 10% improvement in benchmarks vs. previous versions

Yi-Lightning

Efficient training, deployment, and inference

DeepSeek-V3

Remarkably stable training without irrecoverable loss spikes

C. Reliable Foundations

OLMo2

7B: +10.6 pts, 13B: +10.3 pts average; GSM8K: +43.4/+37.8

Llama3

Annealing improved 8B GSM8k by 24.0%, MATH by 6.4%

Hunyuan-Large

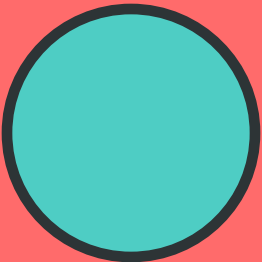
Substantial improvements in downstream task performance

Nemotron4

Better learning from data, significant overall quality improvement

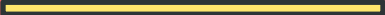


Overall Impact: Mid-training consistently enhances LLM performance across **reasoning**, **efficiency**, and **robustness** dimensions.



07

Future Directions



Open challenges and promising research avenues



Key Open Challenges

Despite growing adoption, mid-training remains **more of an engineering art than a principled science**. Below are key challenges and promising directions.

1

Dynamic Curriculum Design

Develop **adaptive curriculum strategies** that adjust data composition based on the model's evolving capabilities, rather than relying on fixed sampling ratios.

Monitor Signals:

- Loss sharpness
- Gradient variance
- Benchmark outcomes

Transition Strategy:

- Broad → specialized
- Natural → reasoning
- Adaptive mixing

★ **Goal:** Improve efficiency while mitigating overfitting

2

Theoretically Grounded Schedulers

Learning rate schedulers remain **critical yet underexplored**. Future work should unify empirical findings under a theoretical framework.

Adaptive Mechanisms

Schedulers that co-evolve with scale and data

Scale-Aware Design

Generalize across model sizes and regimes

3

Long-Context Progress

Progress hinges on **two fronts**: fixing position-embedding OOD and cutting compute/latency.

Position Functions

General, learnable position functions and index warps

Efficiency

KV-cache compression, token pruning, hybrid stacks

CONCLUSION

Mid-Training: A Distinct Stage in LLM Development

Mid-training has emerged as a **distinct and coherent stage** in the LLM development pipeline, not merely an ad-hoc collection of heuristics. By strategically refining **data distribution**, **learning rate schedules**, and **context length**, models achieve superior reasoning, efficiency, and robustness.



Data Quality

Curated corpora over quantity



Optimization

Stable convergence strategies



Context

Progressive length extension

As the field progresses, **principled approaches to mid-training** will be essential for shaping the next generation of large language models.

Survey Paper

30 Pages

Comprehensive